

Projekt 1 - Raport

Problemy Informatyczne w Transporcie Zbiorowym

Łukasz Trzos

Cele projektu

- Stworzenie efektywnej implementacji wybranych algorytmów wyszukiwania najkrótszej ścieżki w grafie drogowym poznanych na zajęciach
- Stworzenie generatora losowych grafów przypominających z charakteru istniejące grafy drogowe
- Ewaluacja zaimplementowanych algorytmów na wygenerowanych grafach
- Przedstawienie rezultatów w przystępny sposób

W poniższym projekcie korzystałem z narzędzi sztucznej inteligencji, konkretnie były to duże modele językowe, jako wsparcia przy tworzeniu algorytmów oraz środowiska testowego.

Generator grafów

Generator grafów został zaimplementowany w oparciu o podejście geometryczne, w którym wierzchołki są rozmieszczane w przestrzeni dwuwymiarowej, a następnie łączone krawędziami na podstawie odległości euklidesowej.

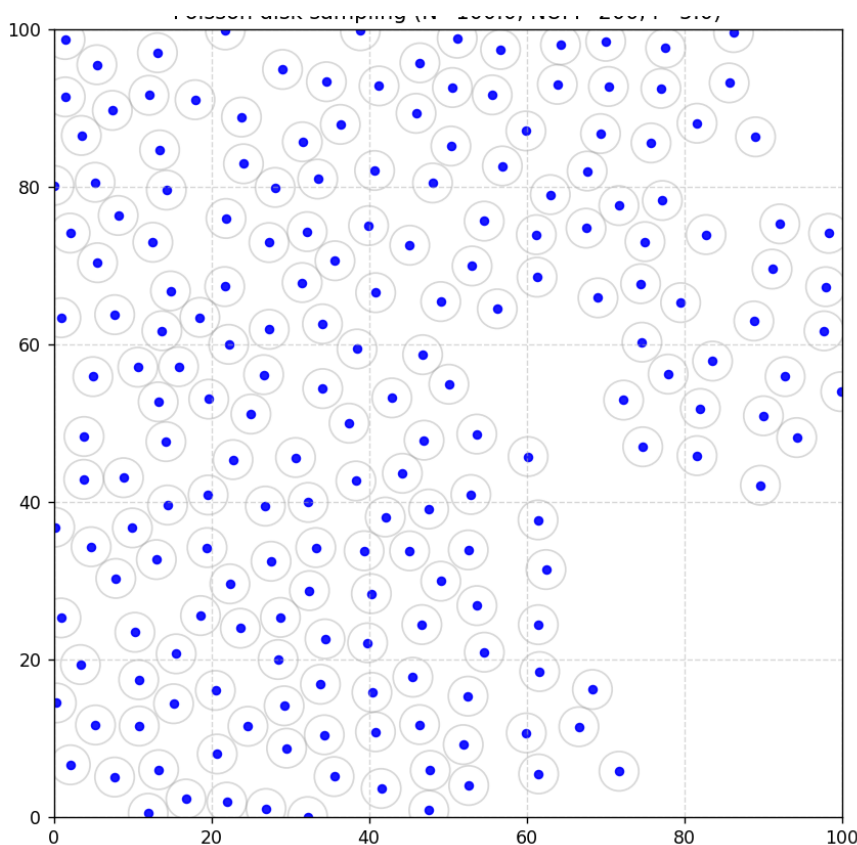
W pierwszym kroku wykorzystywany jest algorytm próbkowania Poissona (Bridson Poisson Disk Sampling), którego celem jest generowanie punktów o równomiernym rozkładzie, przy jednoczesnym zachowaniu minimalnej odległości między nimi. Dzięki temu uzyskujemy strukturę przypominającą rzeczywiste rozmieszczenie skrzyżowań w sieciach drogowych, gdzie punkty nie są skupione losowo, lecz zachowują pewien naturalny dystans.

Minimalna odległość między punktami jest dobierana dynamicznie na podstawie:

$$\text{min_dist} \approx \sqrt{\frac{A}{n \cdot c}}$$

gdzie A oznacza pole obszaru, n liczbę węzłów, a c współczynnik gęstości (empirycznie przyjęty ok. 0.9).

Jeżeli liczba wygenerowanych punktów odbiega od oczekiwanej, algorytm iteracyjnie dostosowuje promień próbkowania lub dokonuje subsamplingu.



Rysunek 1: Przykładowy zbiór punktów wygenerowany metodą próbkowania Poissona

Po wygenerowaniu węzłów budowany jest graf geometryczny. Dla każdego punktu wyszukiwani są sąsiedzi w promieniu R , przy użyciu struktury KDTree, co znacząco przyspiesza obliczenia. Promień R jest dobierany tak, aby uzyskać zadaną średnią liczbę krawędzi wychodzących d :

$$R = \sqrt{\frac{dA}{n\pi}}$$

Dla każdej pary węzłów spełniającej warunek odległości tworzona jest krawędź.

Graf jest modelowany jako graf skierowany, jednak każda krawędź jest symetryzowana, tzn. dla pary węzłów i, j dodawane są dwie krawędzie:

$$(i \rightarrow j) \quad \text{oraz} \quad (j \rightarrow i)$$

Dzięki temu struktura przypomina dwukierunkową sieć drogową. Wagi krawędzi odpowiadają odległości euklidesowej pomiędzy węzłami, z dodatkowym szumem losowym:

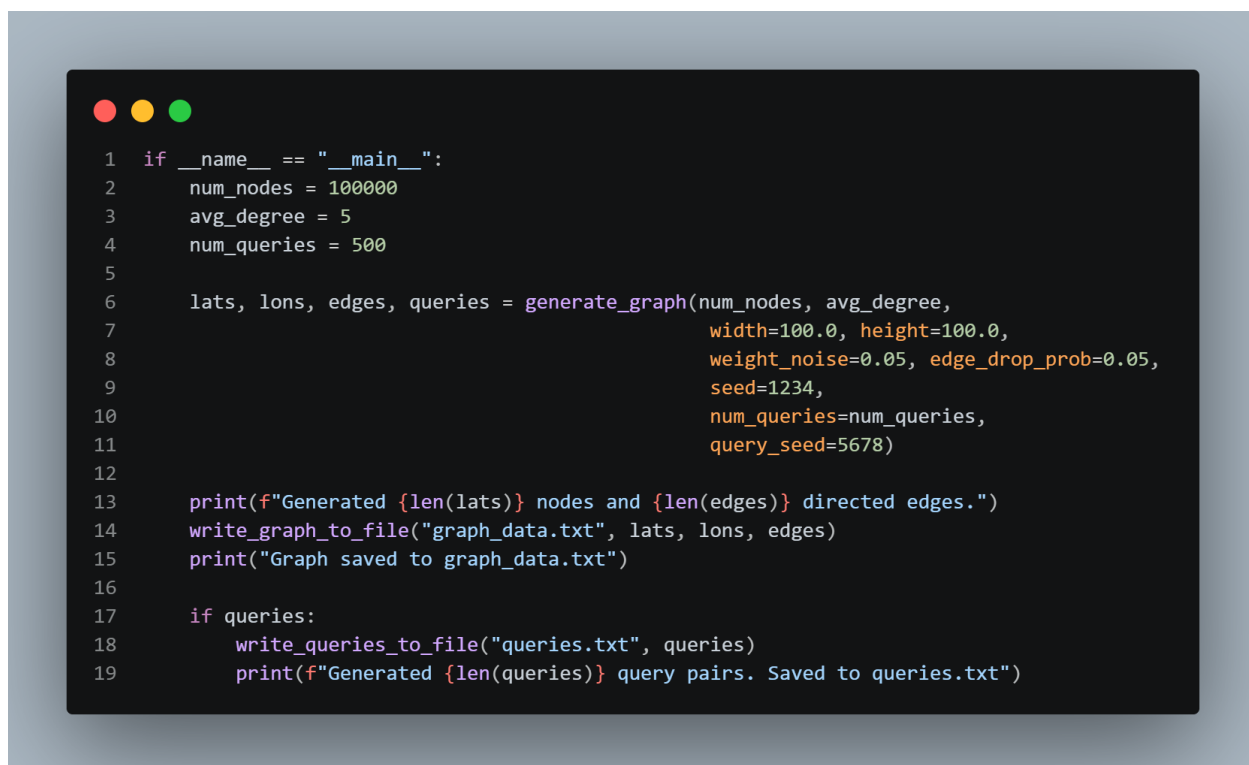
$$w = d(i, j) \cdot (1 + \epsilon)$$

gdzie $\epsilon \in [-\alpha, \alpha]$, a α kontroluje poziom zakłóceń. Wprowadzenie szumu pozwala symulować niedoskonałości rzeczywistych danych drogowych (np. różnice w czasie przejazdu, błędy pomiarowe).

Dodatkowo istnieje możliwość losowego usuwania krawędzi z prawdopodobieństwem p domyślnie ustalonym jako $\frac{1}{20}$, co zwiększa nieregularność grafu.

Generator umożliwia również tworzenie losowych par zapytań typu źródło–cel, które służą do testowania algorytmów wyznaczania najkrótszej ścieżki. Pary te są losowane z warunkiem $s \neq t$, a w przypadku dużej liczby zapytań możliwe jest ich powielanie.

Zastosowanie próbkowania Poissona pozwala uniknąć nierealistycznych skupisk węzłów, co często występuje w prostym losowaniu punktów. Wybór grafu geometrycznego umożliwia naturalną interpretację wag jako odległości, co jest zgodne z charakterem sieci drogowych. Możemy dzięki temu generować grafy względnie rzadkie, gdzie krawędzie prowadzą między bliskoodległymi wierzchołkami.

A screenshot of a code editor with a dark background and light-colored text. The code is written in Python and defines a function to generate a directed graph. It includes parameters for the number of nodes, average degree, width, height, weight noise, edge drop probability, seed, number of queries, and query seed. The code also includes print statements for feedback and file writing functions for saving the graph and queries to text files.

```
1  if __name__ == "__main__":
2      num_nodes = 100000
3      avg_degree = 5
4      num_queries = 500
5
6      lats, lons, edges, queries = generate_graph(num_nodes, avg_degree,
7                                                  width=100.0, height=100.0,
8                                                  weight_noise=0.05, edge_drop_prob=0.05,
9                                                  seed=1234,
10                                                 num_queries=num_queries,
11                                                 query_seed=5678)
12
13     print(f"Generated {len(lats)} nodes and {len(edges)} directed edges.")
14     write_graph_to_file("graph_data.txt", lats, lons, edges)
15     print("Graph saved to graph_data.txt")
16
17     if queries:
18         write_queries_to_file("queries.txt", queries)
19         print(f"Generated {len(queries)} query pairs. Saved to queries.txt")
```

Rysunek 2: Parametry generatora, które możemy łatwo modyfikować celem zróżnicowania grafu wyjściowego

Konstrukcja frameworku

Graf przechowywany jest w strukturze opartej na reprezentacji CSR (Compressed Sparse Row), co zapewnia wysoką wydajność pamięciową oraz szybki dostęp do sąsiadów wierzchołka.

Proces budowy grafu przebiega dwuetapowo:

- w fazie budowania tymczasowego ('add-node', 'add-edge') dane są przechowywane w listach sąsiedztwa,
- po wywołaniu 'build()' struktura zostaje spłaszczona do tablic CSR:

Dodatkowo przechowywana jest również struktura grafu odwróconego, co umożliwia efektywne implementacje algorytmów dwukierunkowych. Po zakończeniu budowy dane tymczasowe są zwalniane, a graf przechodzi w tryb tylko do odczytu.

Wszystkie algorytmy implementowane w systemie dziedziczą po abstrakcyjnej klasie:

```
class Algorithm
{
    public:
        virtual void Preprocess(const Graph& graph) = 0;
        virtual std::vector<int> Query(int source, int target) = 0;
        virtual ~Algorithm() = default;
};
```

Taka konstrukcja zapewnia jednolity interfejs niezależnie od zastosowanej metody wyznaczania ścieżki. Każdy algorytm składa się z dwóch faz:

- **Preprocess(graph)** – opcjonalny etap przygotowania struktur pomocniczych,
- **Query(source, target)** – właściwe obliczenie najkrótszej ścieżki.

Wynikiem działania algorytmu jest wektor identyfikatorów wierzchołków reprezentujący znaną ścieżkę.

System testowy opiera się na osobnym module wykonawczym, który buduje zadany graf i uruchamia wszystkie algorytmy na jego poleceniach.

Dla zapewnienia porównywalności wyników każda implementacja jest najpierw weryfikowana względem rozwiązania referencyjnego (Dijkstra). Następnie mierzony jest czas preprocessingu oraz średni czas każdego zapytania.

Poprawność wyników jest sprawdzana poprzez porównanie kosztu ścieżki zwróconej przez algorytm z wynikiem referencyjnym. Różnice większe niż ustalony próg numeryczny traktowane są jako błąd implementacji.

```
for(auto& algo : algorithms) {
    algo->Preprocess(graph);

    for(int q=0; q<queries.size(); q++) {
        auto Q = queries[q];

        std::vector<int> path =
            algo->Query(Q.first, Q.second);

        double computed_cost = ComputePathCost(graph, path);
        double expected_cost = ground_truth_costs[q];

        const double EPS = 1e-6;
        if(std::fabs(computed_cost - expected_cost) > EPS) {
            std::cout << "WARNING - MISMATCH FOR QUERY NR " << q << " !!!\n";
        }
    }
}
```

Zaimplementowane algorytmy

Wszystkie algorytmy korzystają z jednolitego interfejsu 'Algorithm', który definiuje dwie fazy działania: preprocessing oraz obsługę zapytań.

- **Dijkstra** – klasyczny algorytm najkrótszych ścieżek oparty na kolejce priorytetowej. Stanowi punkt odniesienia (ground truth) do weryfikacji poprawności pozostałych metod.
- **A*** – rozszerzenie algorytmu Dijkstry wykorzystujące funkcję heurystyczną opartą o odległość euklidesową w przestrzeni współrzędnych węzłów. Pozwala na przyspieszenie wyszukiwania poprzez kierowanie eksploracji w stronę celu.
- **BiDirectional Dijkstra** – wariant algorytmu Dijkstry uruchamiany jednocześnie od źródła i celu, z przeszukiwaniem grafu w obu kierunkach. Spotkanie fal wyszukiwania pozwala ograniczyć przestrzeń przeszukiwania.
- **ALT (A* + Landmarks + Triangle inequality)** – algorytm A* wykorzystujący heurystykę opartą na wcześniej wybranych punktach orientacyjnych (landmarkach). Na etapie preprocessingu obliczane są odległości od i do landmarków, co pozwala uzyskać silniejsze oszacowanie kosztu pozostałej drogi.
- **ARC Dijkstra** – algorytm wykorzystujący dekompozycję przestrzenną grafu na regiony. W preprocessing etapie wyznaczane są krawędzie graniczne oraz informacje o ich dostępności między regionami, co pozwala ograniczać przestrzeń przeszukiwania w czasie zapytań.
- **ARC A*** – rozszerzenie ARC Dijkstra o heurystykę A*, która dodatkowo przyspiesza przeszukiwanie poprzez preferowanie węzłów bliższych celowi.
- **ARC ALT** – połączenie podejścia regionowego z heurystyką ALT, łączące redukcję przestrzeni przeszukiwania z silniejszymi estymatami kosztu.

Wyniki i wnioski

Na kolejnej stronie znajdują się tabele zawierające zaobserwowane osiągi czasowe dla różnych rodzin grafów. Wyciągamy następujące wnioski:

Wygenerowane grafy stanowią szczególnie korzystne środowisko dla algorytmu A*. Ponieważ wagi krawędzi są bezpośrednio związane z odległością geometryczną pomiędzy wierzchołkami, zastosowana heurystyka euklidesowa bardzo dobrze aproksymuje rzeczywisty koszt dojścia do celu. Dzięki temu A* znacząco ogranicza przestrzeń przeszukiwania i osiąga wielokrotnie krótsze czasy odpowiedzi niż algorytm Dijkstry, przy zerowym preprocessingu.

Wraz ze wzrostem średniego stopnia wierzchołków rośnie liczba potencjalnych ścieżek, które muszą zostać rozważone przez algorytm Dijkstry. Algorytm ALT wykorzystuje dodatkowe informacje wyznaczone podczas preprocessingu, dzięki czemu ogranicza obszar przeszukiwania. W rezultatach widzimy, że różnica między algorytmami Dijkstry w różnych formach a algorytmem ALT staje się coraz większa.

Tabele z wynikami

Algorytm	Preprocessing [s]	Zestaw zapytań [s]	Pojedyncze zapytanie [s]
Dijkstra	0.000000	16.287186	0.032574
BiDir. Dijkstra	0.014568	12.709538	0.025419
A*	0.000000	5.786187	0.011572
ALT	4.304466	3.624405	0.007249
ARC Dijkstra	213.317991	3.009051	0.006018
ARC A*	209.543570	2.520747	0.005041
ARC ALT	213.136062	1.645070	0.003290

Tabela 1: Porównanie czasów działania algorytmów dla grafu 100k wierzchołków, średni stopień 5

Algorytm	Preprocessing [s]	Zestaw zapytań [s]	Pojedyncze zapytanie [s]
Dijkstra	0.000001	25.646451	0.051293
BiDir. Dijkstra	0.036209	20.119893	0.040240
A*	0.000001	5.632114	0.011264
ALT	6.547879	5.754486	0.011509
ARC Dijkstra	725.468002	5.425049	0.010850
ARC A*	720.525712	3.307349	0.006615
ARC ALT	735.368096	2.179210	0.004358

Tabela 2: Porównanie czasów działania algorytmów dla grafu 100k wierzchołków, średni stopień 12

Algorytm	Preprocessing [s]	Zestaw zapytań [s]	Pojedyncze zapytanie [s]
Dijkstra	0.000001	42.407657	0.084815
BiDir. Dijkstra	0.086801	32.620606	0.065241
A*	0.000001	6.877164	0.013754
ALT	10.554942	6.090626	0.012181
ARC Dijkstra	2203.788339	8.552815	0.017106
ARC A*	2202.118624	4.010661	0.008021
ARC ALT	2199.356128	3.339344	0.006679

Tabela 3: Porównanie czasów działania algorytmów dla grafu 100k wierzchołków, średni stopień 30

Algorytm	Preprocessing [s]	Zestaw zapytań [s]	Pojedyncze zapytanie [s]
Dijkstra	0.000001	98.721946	0.197444
BiDir. Dijkstra	0.094293	77.745221	0.155490
A*	0.000010	33.496250	0.066993
ALT	24.122096	25.473531	0.050947
ARC Dijkstra	3319.984086	18.705657	0.037411
ARC A*	2866.313275	15.421673	0.030843
ARC ALT	2966.149854	9.526865	0.019054

Tabela 4: Porównanie czasów działania algorytmów dla grafu 500k wierzchołków, średni stopień 5